

Rechazo Certificación Metrológica

Usuario: [File: PROMPT_AUDITORIA_RONDA6_CIERRE_SUBHITO_2_2.md]

use.ai: Dictamen: RECHAZADO para certificar el cierre metrológico del Subhito 2.2 en su estado actual. Las mejoras del buffer y de trazabilidad son útiles, pero el filtro propuesto no garantiza eliminar el caudal fantasma, la calibración de permeado no está demostrada y las prestaciones de parada declaradas no se deducen del código.

Es posible mejorar sustancialmente la adquisición **sin modificar ahora el hardware**. No es posible garantizar por software la separación entre dos señales eléctricas idénticas, una producida por flujo y otra por interferencia. Entrego una implementación completa con PCNT para adquisición y diagnóstico, no una promesa físicamente imposible de inmunidad absoluta.

Esta es una revisión documental y de código, no una certificación profesional de seguridad industrial. No he ejecutado el compilador ni realizado mediciones sobre la planta. Además, falta `index_html.h`, por lo que el material suministrado no permite reproducir por sí solo la compilación completa de la aplicación.

1. Filtro temporal: no garantiza rechazar el ruido

1.1. Un contraejemplo basta para invalidar la garantía

Supongamos que la interferencia genera un flanco descendente cada 20 ms:

$$t_{i,20,000} \mu s$$

Entonces:

$$f=50 \text{ Hz}, \quad \frac{dt_{\max}}{dt_{\min}}=1$$

En una ventana con 50 pulsos:

$$t_{\text{último}} - t_{\text{primero}} = 980 \text{ ms}$$

La señal supera todos los filtros:

Blanking: 20 ms es mayor que 1,5 ms.

Dispersión: 1 es menor que 4.

Ocupación: 980 ms es mayor que 30 ms.

Frecuencia: 50 Hz es mayor que 2,5 Hz.

Caudal: con el K de permeado resulta 72,75 mL/min, superior al corte de 30 mL/min.

El firmware acepta esa interferencia como flujo y totaliza aproximadamente:

$$V_{28,\min}$$

$$\frac{50 \cdot 1680}{60 \cdot 687.33}$$

$$2.037 \text{ L}$$

Respuesta a la pregunta 1: no es suficiente, incluso con una interferencia perfectamente periódica y sin ráfagas.

1.2. El blanking es un tiempo muerto, no un filtro de ancho de pulso

Para un tren periódico de entrada de frecuencia f_e y un tiempo muerto no redisparable T_b , un modelo ideal da:

$$f_s = \frac{f_e}{\lceil f_e T_b \rceil}$$

Con $T_b = 1.5 \text{ ms}$:

$$f_e = 1066.7 \text{ Hz} \rightarrow f_s \approx 533.3 \text{ Hz}$$

$$f_e = 4800 \text{ Hz} \rightarrow f_s = 600 \text{ Hz}$$

Los intervalos de salida también pueden ser uniformes y pasar el filtro de dispersión.

Hay una corrección importante respecto del diagnóstico anterior: **el blanking por sí solo no demuestra el origen de los 15 a 65 Hz observados**. Para obtener esas frecuencias debe intervenir la estructura de ráfagas, la modulación, la respuesta eléctrica, la pérdida de flancos u otro mecanismo. La dependencia con RPM es evidencia de acoplamiento con el accionamiento, no una identificación completa de su mecanismo.

1.3. El algoritmo también puede rechazar flujo verdadero

La bomba produce:

$$f_{\text{rodillos}} = \frac{3, \text{RPM}}{60}$$

Entre 20 y 100 RPM, la excitación hidráulica fundamental está entre 1 y 5 Hz. A caudal pequeño, una turbina puede acelerar, detenerse y reiniciar dentro de ese ciclo.

Por tanto, **fluido incompresible no implica velocidad angular uniforme**. La hipótesis $\frac{dt_{\max}}{dt_{\min}} \gg 1$ necesita validación experimental; no es una ley física aplicable a esta instalación.

Además:

Ráfagas uniformes: 20 pulsos separados 2 ms, concentrados en 38 ms, pasan dispersión y ocupación. El estimador informa 500 Hz, aunque el resto del segundo no haya actividad.

Dos o tres pulsos: quedan exceptuados de comprobaciones relevantes.

Silencios: no se evalúan adecuadamente los tiempos anteriores al primer pulso y posteriores al último.

Diagnóstico: `pulsosBrutos()` devuelve pulsos posteriores al blanking, no todos los flancos capturados.

Desbordamiento temporal: `t_ult > t_prim` falla cuando `micros()` cruza su rollover, aproximadamente cada 71,6 minutos.

Fallo oculto: cualquier pulso, incluso uno posteriormente rechazado, reinicia el temporizador de ausencia de señal.

Coherencia de magnitudes: frecuencia, caudal suavizado y volumen corresponden a estimadores distintos y pueden discrepar considerablemente.

1.4. Los datos históricos requieren reconciliación

Con 65 Hz sostenidos durante 28 minutos y el K de permeado declarado:

$$V_{\max} \approx 2.648 \text{ L}$$

Para acumular 10,2 L en ese tiempo se necesita:

$$\overline{Q} = 364.3 \text{ mL/min}, \quad \overline{f} \approx 250.4 \text{ Hz}$$

Por tanto, **10,2 L no se explican únicamente por 15 a 65 Hz con ese K**. Deben revisarse el CSV original, cambios de K, volumen inicial, versiones de firmware y picos no descritos.

La ordenada de 20,98 Hz tampoco demuestra por sí sola EMI: puede reflejar no linealidad, sesgo del patrón, extrapolación o transitorios. El ensayo seco fortalece la hipótesis de interferencia, pero no permite atribuirle automáticamente toda la ordenada ni corregirla simplemente restándola.

2. Circuito RC existente

2.1. Frecuencia de corte y tiempos de transición

Si efectivamente hay dos capacitores de 100 nF:

$$C = 200 \text{ nF}, \quad \tau = 4700 \cdot 200 \cdot 10^{-9} = 0.94 \text{ ms}$$

Para el modelo RC de primer orden:

$$f_c = \frac{1}{2\pi RC} \approx 169.3 \text{ Hz}$$

El tiempo de subida entre 10 % y 90 % es:

$$t_{10-90} = \tau \ln 9 \approx 2.07 \text{ ms}$$

No debe confundirse τ con el tiempo completo de subida.

El segundo capacitor se describe inicialmente sin valor. **Debe verificarse que realmente sea de 100 nF** antes de adoptar estos resultados.

2.2. Puede atenuar interferencia y, simultáneamente, degradar pulsos útiles

Para una perturbación que realmente atraviere ese RC:

$$|H(f)| = \frac{1}{\sqrt{1 + (f/f_c)^2}}$$

A 1067 Hz y 4800 Hz, la atenuación aproximada es de -16,1 dB y -29,1 dB, respectivamente.

Sin embargo, **esa respuesta no describe toda posible interferencia del gabinete**. Una perturbación inyectada directamente en el nodo, un desplazamiento de masa o el acoplamiento sobre el cable pueden tener respuestas diferentes.

Si el sensor es de colector abierto, la subida depende del pull-up y del capacitor; la bajada depende de la capacidad de hundimiento del transistor. No es un filtro simétrico ideal para los pulsos Hall.

Usando los límites lógicos publicados para ESP32, 0,25 y 0,75 veces la alimentación, una carga exponencial tarda aproximadamente:

$$t_{25} = 0.270 \text{ ms}, \quad t_{75} = 1.303 \text{ ms}$$

La permanencia entre esos niveles es de aproximadamente **1,03 ms**. Esos límites especifican niveles garantizados, no los umbrales exactos ni la histéresis efectiva de cada dispositivo.

Espressif, características eléctricas

Una subida lenta **no produce por sí misma oscilaciones**. Combinada con ruido y una histéresis insuficiente, sí puede permitir cruces múltiples de umbral.

Como referencia, una señal ideal con duty de 50 % necesita:

$$\frac{1}{2f} > 1.303 \text{ ms} \Rightarrow f < 384 \text{ Hz}$$

para alcanzar el nivel alto garantizado bajo ese modelo de carga desde cero. No es una especificación universal del montaje, pero muestra que el RC puede afectar frecuencias útiles.

2.3. Mitigación sin cambiar componentes

Recomiendo **PCNT con filtro de glitches**, sin blanking de 1500 µs, acompañado de caracterización de la señal.

También puede evaluarse una validación de permanencia en alto y bajo, pero sus umbrales deben derivarse de anchos de pulso medidos. Un antirrebote de milisegundos seleccionado arbitrariamente puede eliminar pulsos verdaderos.

Ninguna técnica digital:

Recupera pulsos que no alcanzan los niveles lógicos.

Corrige sobretensión del GPIO.

Repara una referencia de masa inadecuada.

Distingue interferencia de un pulso Hall eléctricamente idéntico.

Verificar el tipo y nivel de salida del sensor es obligatorio. Una resistencia a 3,3 V no convierte por sí sola una salida push-pull de 5 V en una señal segura. Hay ofertas de YF-S401 que especifican salida de 4,7 V con alimentación de 5 V; eso obliga a identificar el ejemplar real, no a asumir colector abierto. [TinyTronics](#)

3. PCNT frente a ISR

Sí recomiendo migrar a PCNT, pero como mejora de adquisición, no como certificador de origen hidráulico.

El ESP32 original dispone de **8 unidades PCNT, con dos canales por unidad**, no simplemente ocho canales. Los canales de una misma unidad comparten contador. [Espressif, PCNT](#)

A 80 MHz:

$$t_{\max} = \frac{1023}{80 \times 10^6} = 12.7875 \mu s$$

Ese filtro puede eliminar glitches cortos; **no elimina automáticamente ruido de 20 μs, 1 ms o pulsos periódicos de 50 Hz**. La API moderna expresa el umbral en nanosegundos y gestiona la dependencia del reloj mediante un bloqueo de gestión de energía. [Espressif, filtro PCNT](#)

La implementación de la sección 8 utiliza, por sensor:

Una unidad sin filtro: diagnóstico de flancos digitales brutos.

Una unidad con filtro de 10 μs: adquisición de pulsos candidatos.

Conteo continuo: sin detener y limpiar el contador cada segundo.

Compensación de overflow: mediante acumulación y watch points.

Tiempo real transcurrido: para calcular frecuencia media de ventana.

Estado de calidad explícito: independiente del valor numérico.

Las dos unidades no se leen exactamente al mismo instante. Sus diferencias de una ventana no deben interpretarse como una clasificación exacta, pulso por pulso, de lo descartado.

4. Metrología del permeado

4.1. La calibración matemática no recupera información inexistente

Si varios caudales distintos producen cero pulsos por fricción estática:

$$Q_1 \neq Q_2, \quad N(Q_1) = N(Q_2) = 0$$

no existe una función inversa única que recupere ambos caudales a partir del conteo.

La turbina puede ser aceptable fuera del rango comercial **solo si se demuestra experimentalmente**, para ese ejemplar y montaje, una respuesta repetible, suficientemente sensible y con incertidumbre compatible con el objetivo de la tesis.

Una especificación comercial consultada declara 0,3 a 6 L/min y $\pm 10\%$, además de un factor de 98 Hz/(L/min). Esto no demuestra que sus sensores tengan ese factor, pero sí que ni 196,50 ni 687,33 deben denominarse automáticamente "calibración de fábrica" del YF-S401. [TinyTronics](#)

Los 30 mL/min del firmware **no constituyen un límite de cuantificación demostrado**.

Además, los cortes actuales equivalen a:

$$f_{\min,\mathrm{alim}}$$

$$\max(2.5, 0.030 \cdot 196.50)$$

$$5.895\,\mathrm{Hz}$$

$$f_{\min,\mathrm{perm}}$$

$$\max(2.5, 0.030 \cdot 687.33)$$

$$20.620\,\mathrm{Hz}$$

Un ruido de 25 Hz en permeado ya los supera.

4.2. La gravimetría no exige obligatoriamente HX711
El tribunal puede objetar una incertidumbre no demostrada, pero **no se puede anticipar que exigirá un componente específico**.

Una balanza contrastada, un recipiente y una base de tiempo adecuada permiten establecer una referencia independiente. Una celda de carga con HX711 es una opción de automatización, no una condición universal de validez.

La ecuación volumétrica correcta incorpora la densidad:

$$Q_{\mathrm{mL/min}}$$

$$\frac{60 \cdot \Delta m_{\mathrm{g}}}{\rho_{\mathrm{g/mL}} \cdot \Delta t_{\mathrm{s}}}$$

Debe controlarse drenaje residual, evaporación, vibración, resolución de balanza y sincronización.

4.3. Procedimiento para Antonella
Propongo este protocolo, sin cambiar inicialmente el circuito:

Identificar el montaje. Registrar sensor, orientación, tensión, temperatura, mangueras, capacitor efectivo y versión de firmware. Desactivar la autocalibración por RPM.

Ensayar ceros. Sensor seco y, por separado, sensor lleno sin flujo. Probar motor detenido y los cinco regímenes solicitados. Incluir arranque, frenado, inversión y exportación CSV.

Medir señal eléctrica. Observar amplitud, anchos y transiciones en el GPIO respecto de la masa del ESP32. Comparar sensor seco y flujo real.

Ensayar caudales de referencia. Cubrir 20, 30, 50, 75 y 100 mL/min, si el banco los permite, y puntos de transición hacia 200 y 300 mL/min. Realizar al menos tres repeticiones ascendentes y descendentes por punto.

Sincronizar conteo y recolección. Registrar pulsos acumulados durante exactamente el intervalo de masa o volumen de referencia. No calibrar con una frecuencia instantánea tomada después de recolectar.

Elegir duración por incertidumbre. Como criterio inicial, buscar al menos 100 pulsos por punto y una masa neta suficientemente superior a la resolución de la balanza. Si no hay pulsos pese a existir flujo gravimétrico, registrar esa falla de sensibilidad.

Calcular el factor por punto.

$$K_i = \frac{N_i}{60 \cdot V_{i, \text{L}}}$$

Examinar dependencia de K con caudal, temperatura, orientación y sentido del barrido.

Validar con datos independientes. No usar todos los puntos para ajustar y después presentar el mismo conjunto como validación. Evaluar sesgo, repetibilidad, histéresis y rango de medición.

Definir el rango aceptado. Publicar error e incertidumbre máximos antes de aceptar el instrumento. Fuera de ese rango informar "no cuantificable", no cero.

Mantener la referencia externa. Si la turbina no supera la validación de 20 a 100 mL/min, usar gravimetría para permeado y conservar la turbina como indicador experimental.

Como aproximación para componentes independientes:

$$\left(\frac{u(Q)}{Q}\right)^2 \approx \left(\frac{u(\Delta m)}{\Delta m}\right)^2 + \left(\frac{u(\rho)}{\rho}\right)^2 + \left(\frac{u(\Delta t)}{\Delta t}\right)^2$$

A esto deben añadirse las contribuciones del procedimiento, no solo las resoluciones instrumentales.

4.4. Inconsistencia entre KUF y caudal esperado

Usando exclusivamente los parámetros declarados:

$$Q = K_{\text{UF}} \cdot \text{TMP}$$

A 0,50 bar:

$$\text{TMP} = 0.75 \text{ bar} \Rightarrow Q = 0.75 \cdot K_{\text{UF}} = 0.75 \cdot 1.33 = 1.00 \text{ L/min}$$

$\frac{1}{10} \approx 0.1$

$Q \approx \frac{73 \cdot 375}{60} = 456.3 \text{ mL/min}$

A 0,20 bar resulta aproximadamente **182,5 mL/min**.

Esto no predice necesariamente el comportamiento real con agua y efluentes: el KUF nominal tiene condiciones de determinación particulares. Sí demuestra que **20 a 100 mL/min no se deduce de los parámetros nominales indicados** sin información adicional.

5. Observaciones bloqueantes del resto del firmware

5.1. Parada y seguridad

El cálculo:

$100/70 = 1.4286 \text{ s}$

es correcto como tiempo ideal de rampa. No garantiza el tiempo desde la orden hasta la detención física.

`server.handleClient()` y la exportación comparten el hilo con `bomba.tick()`. Una transmisión lenta puede retrasar el control. `yield()` permite ejecutar otras tareas, pero no ejecuta por sí solo el siguiente `bomba.tick()` del `loop`.

Asimismo:

RPM estimadas: `_actual` representa una orden, no velocidad medida.

Emergencia por software: `paradaEmergencia()` no constituye un circuito independiente de parada de emergencia.

ENA: falta verificar polaridad, corriente de entrada, compatibilidad con 3,3 V y tiempo de respuesta del driver real.

Desenergización: retirar torque no implica detener instantáneamente el rotor.

Presión: los límites definidos en `config.h` no están conectados a ninguna medición ni interbloqueo.

LEDC: no se comprueban los resultados de configuración.

Reposo: duty cero con driver habilitado no significa necesariamente "motor libre"; puede mantener corriente de retención.

La afirmación **"desenergiza físicamente en menos de 1 ms"** no está demostrada.

Mientras no se verifiquen protecciones de presión, no corresponde interpretar el software como autorización para ensayar un circuito presurizado sin protección independiente.

5.2. Buffer, sesiones y régimen

El buffer circular es una mejora correcta. Sin embargo, **O(1) no significa conservación indefinida**.

A 10 segundos por muestra, 600 registros cubren aproximadamente 100 minutos; las muestras

A 10 segundos por muestra, 600 registros cubren aproximadamente 100 minutos, las muestras adicionales de inicio y cierre reducen esa duración. Al llenarse, se siguen perdiendo datos antiguos. Debe registrarse explícitamente el número de muestras sobrescritas.

Los 38 KB/s de movimiento anteriores eran un gasto evitable, pero no prueban por sí solos una limitación crítica de un ESP32. El problema metrológico principal era la pérdida silenciosa de historia.

Otras observaciones:

Régimen: proximidad entre RPM ordenadas no demuestra estabilización hidráulica.

Cambios pequeños: sucesivos cambios inferiores a 1,5 RPM pueden alterar mucho la consigna sin segmentar el ensayo.

Cierre temprano: el ensayo termina cuando `_enMarcha` cambia a falso, antes de finalizar la rampa y de actualizar todo el volumen.

CSV: no contiene conteos brutos, calidad, tiempo efectivo de ventana ni contador de pérdida.

Muestreo: guardar una lectura cada 10 s no equivale a promediar los 10 s anteriores.

Identificadores: `uint16_t` aplaza el overflow, no lo impide.

JSON: truncar una cadena de `snprintf` no conserva un documento válido.

Los recorridos de `bufferLog[0..bufferCount)` para contar registros por ensayo no son, por sí mismos, un error del buffer circular: antes de llenarse son las posiciones ocupadas; después están ocupadas todas. El orden cronológico sí requiere el índice circular usado en la exportación.

5.3. Calibración y balance

La autocalibración por RPM es circular si el objetivo es contrastar el caudal real de la bomba:

$$K_{\mathrm{nuevo}}$$

$$\frac{f}{\mathrm{RPM}} \cdot V_{\mathrm{rev}}$$

Después de aplicar ese K, el sensor tenderá a concordar con el modelo usado como patrón. No prueba independencia ni exactitud.

Además, `handleCalibrarRpmQ()` mezcla caudal de probeta con frecuencia actual, sin garantizar simultaneidad, y permite modificar parámetros durante una adquisición.

El balance:

$$Q_r = Q_a - Q_p$$

es una inferencia bajo hipótesis de conservación y acumulación despreciable. **Recortar a cero un resultado negativo oculta una inconsistencia**, no la resuelve. Tampoco un balance construido así constituye una validación independiente de ambos sensores.

5.4. Darcy

Las conversiones de LMH y el factor de normalización de viscosidad están correctamente

Las conversiones de L/min y el factor de normalización de viscosidad están correctamente orientados:

$$J_{20} = J_T \frac{\mu_T}{\mu_{20}}$$

Pero hay limitaciones:

TMP supuesta: 0,20 bar no debe registrarse como presión medida.

Temperatura fuera de rango: limitarla silenciosamente a 5 o 60 °C oculta una entrada inválida.

Calibración de Rm: no comprueba finitud de todos los datos ni del resultado antes de aceptarlo.

Temperatura variable: ajustar una única pendiente y usar una sola viscosidad exige temperatura controlada.

Polarización: atribuir toda caída de flujo a resistencia ignora posibles contribuciones osmóticas.

A partir del KUF, área y viscosidad declarados, la resistencia equivalente nominal sería aproximadamente:

$$R_m = \frac{A}{\mu L_{p, \text{total}}} \approx 1.44 \times 10^{13} \text{ m}^{-1}$$

No coincide con $2.09 \times 10^{13} \text{ m}^{-1}$. Debe documentarse la procedencia de ambos valores; no sustituirlos automáticamente sin comprobar condiciones.

6. Qué demuestra la prueba seca

Pasarla es necesario, pero no suficiente para cerrar el subhito.

Cero eventos durante 300 s no demuestra una tasa universal exactamente nula. Bajo un modelo Poisson estacionario, el límite superior unilateral aproximado del 95 % es:

$$\lambda_{95}$$

$$\frac{-\ln(0.05)}{300} \approx 0.00999 \text{ s}^{-1}$$

Con el K de permeado, eso equivaldría aproximadamente a 0,0145 mL/min. Es una conversión ilustrativa: el ruido puede no ser Poisson y ese K aún no está validado.

Debe distinguirse:

Cero flancos brutos: criterio estricto solicitado.

Cero pulsos posteriores al filtro: aceptación del acondicionamiento digital bajo las condiciones ensayadas.

Cero caudal mostrado: no prueba ninguna de las anteriores si el programa lo fuerza a cero.

La prueba también debe demostrar que **sí se detecta el flujo real mínimo**. Un sensor desconectado o una entrada permanentemente fija pasan fácilmente una prueba de cero.

7. Arquitectura propuesta y alcance

La propuesta siguiente reemplaza los dos archivos de caudalímetros y conserva sus métodos

La propuesta siguiente reemplaza los dos drivers de caudalímetros y conserva sus métodos públicos originales. Añade diagnósticos y estados de calidad.

No incluye un clasificador ficticio de EMI. Un pulso suficientemente ancho puede atravesar PCNT aunque sea interferencia.

Decisiones de implementación:

Sin blanking de 1500 μ s, dispersión arbitraria ni deadband de 30 mL/min.

Frecuencia de ventana: $f = N / \Delta t$.

Volumen por conteo: $\Delta V = N / (60K)$.

Sin EMA metrológica: un eventual suavizado de pantalla debe mantenerse separado.

Sin falsas calibraciones: al iniciar, los valores numéricos son provisionales.

Modo seco diagnóstico: enclava la detección de cualquier flanco bruto y no totaliza como líquido.

Errores explícitos: ventanas mayores de 5 s, fallos PCNT y agotamiento del acumulador invalidan la adquisición.

Límite de sesión: se detiene antes de mil millones de flancos acumulados. No presupone que el acumulador de la API sea infinito.

Sin reinicio entre ventanas: no introduce huecos periódicos por `stop/clear/start`.

Se utilizan las funciones de la API moderna documentada para ESP-IDF 5.1, incluida la compensación de overflow mediante watch points. [Espressif, API PCNT 5.1](#)

Compatibilidad prevista: ESP32 original, Arduino-ESP32 3.x, con `driver/pulse_cnt.h`. El código está completo, pero su compilación y funcionamiento deben verificarse con la versión exacta instalada; no los presento como ensayos aquí.

8. Código completo

```
caudalimetro.h

#pragma once

#include <Arduino.h>
#include <cmath>
#include <cstdint>
#include "config.h"
#include "driver/pulse_cnt.h"

class Caudalimetro {
public:
    enum class Calidad : uint8_t {
        NO_INICIADO,
        ERROR_ADQUISICION,
        PRUEBA_SECA,
        RUIDO_EN_SECO,
        SIN_PULSOS,
    };
};
```

```

    NO_CALIBRADO,
    FUERA_DE_RANGO,
    VALIDO
};

Caudalimetro(uint8_t pin, float k, const char* nombre,
             bool esAlimentacion = true);

Caudalimetro(const Caudalimetro&) = delete;
Caudalimetro& operator=(const Caudalimetro&) = delete;

void begin();
void actualizar(float dt_s, bool bombaEmpuja);

// Valores numericos provisionales salvo calidad() == VALIDO.
float caudal_mLmin() const { return _q; }
float caudal_Lmin() const { return _q / 1000.0f; }
float frecuencia_Hz() const { return _f; }

// Total provisional. Consultar volumenCompleto() y
calibracionValidada().
float volumen_L() const { return static_cast<float>(_vol); }
void resetVolumen();

// Compatibilidad con el SCADA existente: false solo con medicion
valida.
bool sinSenal() const { return !medicionValida(); }

const char* nombre() const { return _nombre; }
bool esAlimentacion() const { return _esAlimentacion; }

void setK(float nuevoK);
float getK() const { return _k; }

// Flancos capturados por PCNT sin filtro, durante la ultima ventana.
uint32_t pulsosBrutos() const { return _rawVentana; }

// Flancos que pasaron el filtro PCNT de 10 us.
uint32_t pulsosFiltrados() const { return _filVentana; }

uint64_t pulsosBrutosTotal() const { return _rawTotal; }
uint64_t pulsosFiltradosTotal() const { return _filTotal; }

double ventana_s() const { return _ventana_s; }

Calidad calidad() const { return _calidad; }
const char* calidadTexto() const;

bool medicionValida() const {
    return _calidad == Calidad::VALIDO;
}

bool adquisicionOperativa() const {
    return _iniciado && !_error;
}

bool faltaPulsosAlimentacion() const { return _sinFlujoAlim; }
bool volumenCompleto() const { return _volCompleto; }

```

```

bool volumenCompleto() const { return _volCompleto; }
bool calibracionValidada() const { return _calibrado; }
bool ruidoDetectadoEnSeco() const { return _ruidoSeco; }

// Habilitar solo despues de una validacion externa documentada.
bool validarCalibracion(float qMin_mLmin, float qMax_mLmin);

// Cada activacion inicia una prueba nueva, incluso si ya estaba
activa.
void setModoSeco(bool activo);

private:
struct Contador {
    pcnt_unit_handle_t unidad = nullptr;
    pcnt_channel_handle_t canal = nullptr;
    bool _habilitado = false;
    bool arrancado = false;
    int anterior = 0;
};

static constexpr int LIMITE_HW = 30000;
static constexpr int LIMITE_SESION = 1000000000;
static constexpr uint32_t FILTRO_NS = 10000;
static constexpr int64_t MAX_VENTANA_US = 5000000;

bool crearContador(Contador& c, bool filtrar);
void pararContador(Contador& c);
void marcarError();

const uint8_t _pin;
float _k;
const char* _nombre;
const bool _esAlimentacion;

Contador _bruto;
Contador _filtrado;

bool _intentado = false;
bool _iniciado = false;
bool _error = false;
bool _calibrado = false;
bool _modoSeco = false;
bool _ruidoSeco = false;
bool _sinFlujoAlim = false;
bool _volCompleto = true;

float _qMin = 0.0f;
float _qMax = Q_MAX_FISICO_MLMIN;
float _q = 0.0f;
float _f = 0.0f;

double _vol = 0.0;
double _sinPulsos_s = 0.0;
double _ventana_s = 0.0;

int64_t _tAnterior_us = 0;

uint32_t rawVentana = 0;

```

```

uint32_t _filVentana = 0;
uint64_t _rawTotal = 0;
uint64_t _filTotal = 0;

Calidad _calidad = Calidad::NO_INICIADO;
};

```

caudalimetro.cpp

```

#include "caudalimetro.h"

#include "driver/gpio.h"
#include "esp_timer.h"
#include "sdkconfig.h"

#if !defined(CONFIG_IDF_TARGET_ESP32)
#error "Implementacion dimensionada para ESP32 original: requiere 4 unidades PCNT."
#endif

Caudalimetro::Caudalimetro(uint8_t pin, float k,
                           const char* nombre, bool esAlimentacion)
    : _pin(pin),
      _k(k),
      _nombre(nombre),
      _esAlimentacion(esAlimentacion) {}

bool Caudalimetro::crearContador(Contador& c, bool filtrar) {
    pcnt_unit_config_t uc{};
    uc.low_limit = -LIMITE_HW;
    uc.high_limit = LIMITE_HW;
    uc.flags.accum_count = 1;

    if (pcnt_new_unit(&uc, &c.unidad) != ESP_OK) {
        return false;
    }

    pcnt_chan_config_t cc{};
    cc.edge_gpio_num = static_cast<int>(_pin);
    cc.level_gpio_num = -1;

    if (pcnt_new_channel(c.unidad, &cc, &c.canal) != ESP_OK) {
        return false;
    }

    // Flanco ascendente: no contar.
    // Flanco descendente: incrementar.
    if (pcnt_channel_set_edge_action(
        c.canal,
        PCNT_CHANNEL_EDGE_ACTION_HOLD,
        PCNT_CHANNEL_EDGE_ACTION_INCREASE) != ESP_OK) {
        return false;
    }

    if (pcnt_channel_set_level_action(
        c.canal,

```

```

        PCNT_CHANNEL_LEVEL_ACTION_KEEP,
        PCNT_CHANNEL_LEVEL_ACTION_KEEP) != ESP_OK) {
    return false;
}

if (filtrar) {
    pcnt_glitch_filter_config_t fc{};
    fc.max_glitch_ns = FILTRO_NS;
    if (pcnt_unit_set_glitch_filter(c.unidad, &fc) != ESP_OK) {
        return false;
    }
} else {
    if (pcnt_unit_set_glitch_filter(c.unidad, nullptr) != ESP_OK) {
        return false;
    }
}

if (pcnt_unit_add_watch_point(c.unidad, LIMITE_HW) != ESP_OK) {
    return false;
}

if (pcnt_unit_add_watch_point(c.unidad, -LIMITE_HW) != ESP_OK) {
    return false;
}

if (pcnt_unit_enable(c.unidad) != ESP_OK) {
    return false;
}
c.habilitado = true;

if (pcnt_unit_clear_count(c.unidad) != ESP_OK) {
    return false;
}

c.anterior = 0;
return true;
}

void Caudalimetro::pararContador(Contador& c) {
    if (c.unidad && c.arrancado) {
        pcnt_unit_stop(c.unidad);
        c.arrancado = false;
    }
}

void Caudalimetro::marcarError() {
    pararContador(_bruto);
    pararContador(_filtrado);

    _error = true;
    _calidad = Calidad::ERROR_ADQUISICION;
    _volCompleto = false;
    _q = 0.0f;
    _f = 0.0f;
    _rawVentana = 0;
    _filVentana = 0;
}

```

```

void Caudalimetro::begin() {
    // Objetos de vida estatica: una sola inicializacion por arranque.
    if (_intentado) {
        return;
    }
    _intentado = true;

    if (!std::isfinite(_k) || _k <= 0.1f) {
        marcarError();
        return;
    }

    pinMode(_pin, INPUT);

    if (!crearContador(_bruto, false) ||
        !crearContador(_filtrado, true)) {
        marcarError();
        return;
    }

    // El montaje utiliza pull-up externo.
    if (gpio_set_pull_mode(static_cast<gpio_num_t>(_pin),
                          GPIO_FLOATING) != ESP_OK) {
        marcarError();
        return;
    }

    if (pcnt_unit_start(_bruto.unidad) != ESP_OK) {
        marcarError();
        return;
    }
    _bruto.arrancado = true;

    if (pcnt_unit_start(_filtrado.unidad) != ESP_OK) {
        marcarError();
        return;
    }
    _filtrado.arrancado = true;

    // Baseline posterior al arranque: descarta el transitorio inicial.
    int raw = 0;
    int fil = 0;

    if (pcnt_unit_get_count(_bruto.unidad, &raw) != ESP_OK ||
        pcnt_unit_get_count(_filtrado.unidad, &fil) != ESP_OK) {
        marcarError();
        return;
    }

    _bruto.anterior = raw;
    _filtrado.anterior = fil;
    _tAnterior_us = esp_timer_get_time();

    _iniciado = true;
    _calidad = _modoSeco
        ? Calidad::PRUEBA_SECA
        : Calidad::NO_CALIBRADO;
}

```

```

}

void Caudalimetro::setK(float nuevoK) {
    if (!std::isfinite(nuevoK) || nuevoK <= 0.1f) {
        return;
    }

    if (nuevoK != _k) {
        // Cambiar K exige una nueva validacion externa.
        // El volumen previo no se recalcula retroactivamente.
        _k = nuevoK;
        _calibrado = false;

        if (_filTotal != 0) {
            _volCompleto = false;
        }

        if (!_error && !_modoSeco) {
            _calidad = Calidad::NO_CALIBRADO;
        }
    }
}

bool Caudalimetro::validarCalibracion(float qMin_mLmin,
                                       float qMax_mLmin) {
    if (!std::isfinite(_k) || _k <= 0.1f ||
        !std::isfinite(qMin_mLmin) ||
        !std::isfinite(qMax_mLmin) ||
        qMin_mLmin <= 0.0f ||
        qMax_mLmin <= qMin_mLmin ||
        qMax_mLmin > Q_MAX_FISICO_MLMIN) {
        return false;
    }

    _qMin = qMin_mLmin;
    _qMax = qMax_mLmin;
    _calibrado = true;

    // La calidad de una muestra cambia en actualizar(), no aqui.
    return true;
}

void Caudalimetro::resetVolumen() {
    _vol = 0.0;

    // No se borran contadores PCNT ni totales de diagnostico.
    // El origen es la ultima frontera de adquisicion procesada.
    _volCompleto = _iniciado && !_error;
}

void Caudalimetro::setModoSeco(bool activo) {
    _modoSeco = activo;

    if (activo) {
        _ruidoSeco = false;
    }
}

~ = 0.0f;

```

```

    _q = 0.0f;
    _f = 0.0f;

    if (!_error) {
        _calidad = activo
            ? Calidad::PRUEBA_SECA
            : Calidad::NO_CALIBRADO;
    }
}

void Caudalimetro::actualizar(float dt_s, bool bombaEmpuja) {
    // Compatibilidad con la API anterior.
    // La base temporal efectiva es esp_timer_get_time().
    (void)dt_s;

    if (!_iniciado || _error) {
        _q = 0.0f;
        _f = 0.0f;
        return;
    }

    int raw = 0;
    int fil = 0;

    if (pcnt_unit_get_count(_bruto.unidad, &raw) != ESP_OK ||
        pcnt_unit_get_count(_filtrado.unidad, &fil) != ESP_OK) {
        marcarError();
        return;
    }

    const int64_t ahora_us = esp_timer_get_time();
    const int64_t elapsed_us = ahora_us - _tAnterior_us;

    // No aceptar una ventana cuya integridad temporal no se garantiza.
    // Un error requiere reinicio controlado del equipo.
    if (elapsed_us <= 0 || elapsed_us > MAX_VENTANA_US ||
        raw < _bruto.anterior || fil < _filtrado.anterior ||
        raw >= LIMITE_SESION || fil >= LIMITE_SESION) {
        marcarError();
        return;
    }

    const uint32_t nr =
        static_cast<uint32_t>(raw - _bruto.anterior);
    const uint32_t nf =
        static_cast<uint32_t>(fil - _filtrado.anterior);

    _bruto.anterior = raw;
    _filtrado.anterior = fil;
    _tAnterior_us = ahora_us;

    _rawVentana = nr;
    _filVentana = nf;
    _rawTotal += nr;
    _filTotal += nf;
    _ventana_s = static_cast<double>(elapsed_us) * 1.0e-6;

    const double f = static_cast<double>(nf) / _ventana_s;

```

```

const double q = 1000.0 * f / static_cast<double>(_k);

if (!std::isfinite(f) || !std::isfinite(q)) {
    marcarError();
    return;
}

// Diagnostico de ausencia, no identificacion de cable cortado.
if (bombaEmpuja && _esAlimentacion && nf == 0) {
    _sinPulsos_s += _ventana_s;
} else {
    _sinPulsos_s = 0.0;
}
_sinFlujoAlim = (_sinPulsos_s >= 5.0);

// Conservar frecuencia observada incluso durante una prueba seca.
_f = static_cast<float>(f);

if (_modoSeco) {
    if (nr != 0 || nf != 0) {
        _ruidoSeco = true;
    }

    _q = 0.0f;
    _calidad = _ruidoSeco
        ? Calidad::RUIDO_EN_SECO
        : Calidad::PRUEBA_SECA;
    return;
}

_q = static_cast<float>(q);

if (q > static_cast<double>(Q_MAX_FISICO_MLMIN)) {
    // No ocultar la frecuencia ni el caudal provisional observado.
    // No integrar una ventana implausible.
    _volCompleto = false;
    _calidad = Calidad::FUERA_DE_RANGO;
    return;
}

// Total por pulsos, independiente de filtros de presentacion.
_vol += static_cast<double>(nf) /
    (60.0 * static_cast<double>(_k));

if (nf == 0) {
    // Ausencia de pulsos no prueba caudal hidraulico exactamente cero.
    _volCompleto = false;
    _calidad = Calidad::SIN_PULSOS;
} else if (!_calibrado) {
    _volCompleto = false;
    _calidad = Calidad::NO_CALIBRADO;
} else if (q < _qMin || q > _qMax) {
    _volCompleto = false;
    _calidad = Calidad::FUERA_DE_RANGO;
} else {
    _calidad = Calidad::VALIDO;
}

```

```

}

const char* Caudalimetro::calidadTexto() const {
    switch ( _calidad ) {
        case Calidad::NO_INICIADO:
            return "NO_INICIADO";
        case Calidad::ERROR_ADQUISICION:
            return "ERROR_ADQUISICION";
        case Calidad::PRUEBA_SECA:
            return "PRUEBA_SECA";
        case Calidad::RUIDO_EN_SECO:
            return "RUIDO_EN_SECO";
        case Calidad::SIN_PULSOS:
            return "SIN_PULSOS";
        case Calidad::NO_CALIBRADO:
            return "NO_CALIBRADO";
        case Calidad::FUERA_DE_RANGO:
            return "FUERA_DE_RANGO";
        case Calidad::VALIDO:
            return "VALIDO";
    }
    return "DESCONOCIDO";
}

```

Condiciones de uso del reemplazo

Los valores numéricos se conservan para diagnóstico y compatibilidad, incluso cuando no están validados. Eso evita introducir `NaN` en el JSON actual, pero obliga a no confundir "número disponible" con "medición certificada".

`sinSenal()` pasa a significar "no hay medición válida" para que `alim_ok` y `perm_ok` no indiquen éxito durante una adquisición no calibrada. La causa concreta está en `calidadTexto()`.

Los totales son provisionales y pueden ser incompletos. `resetVolumen()` **no recupera validez perdida ni corrige datos anteriores**. Debe ejecutarse en una frontera de adquisición, después de `actualizar()`, con la instalación en condición conocida.

Los objetos están previstos para vida estática, como en el firmware original. Un fallo de inicialización o de adquisición queda enclavado y requiere un reinicio controlado. La implementación no intenta reconfigurar periféricos mientras la planta continúa operando.

La clase debe usarse desde **una única tarea propietaria**. Si luego se separa adquisición del servidor, compartir sus datos exige snapshots protegidos; no basta con llamar simultáneamente sus métodos desde dos tareas.

9. Integración inmediata y pruebas del código

9.1. Prueba seca sin manipular el resultado

En el ensayo dedicado, activar el modo seco antes del intervalo de evaluación:

```
sensorPermeado.setModoSeco(true);
```

Después de cada adquisición, registrar ambas observaciones:

```
Serial.printf(
    "[PERMEADO] raw=%lu filtrados=%lu dt=%.6f "
    "f=%.3f estado=%s ruido_seco=%s\n",
    static_cast<unsigned long>(sensorPermeado.pulsosBrutos()),
    static_cast<unsigned long>(sensorPermeado.pulsosFiltrados()),
    sensorPermeado.ventana_s(),
    sensorPermeado.frecuencia_Hz(),
    sensorPermeado.calidadTexto(),
    sensorPermeado.ruidoDetectadoEnSeco() ? "SI" : "NO"
);
```

El criterio de aprobación es el conteo, no `caudal_mLmin()`, que en este modo es cero deliberadamente porque el operador ha declarado que no hay líquido.

Antes de activar o desactivar el modo, cerrar la ventana anterior con `actualizar()`. No cambiar de modo a mitad de una recolección metrológica.

9.2. Detener la autocalibración circular

Hasta disponer de referencia independiente, reemplazar completamente

`handleIniciarAutoCal()` por:

```
void handleIniciarAutoCal() {
    autoCalibrando = false;
    autoCalMuestras = 0;
    autoCalSumFrecAlim = 0.0f;
    autoCalSumFrecPerm = 0.0f;
    autoCalMensaje = "Deshabilitada: requiere patron independiente.";

    server.send(
        409,
        "application/json",
        "{\"status\":\"error\", \"
        \"msg\":\"Calibracion por RPM deshabilitada; \"
        \"utilizar referencia gravimetrica o volumetrica.\"}";
    );
}
```

No llamar a `validarCalibracion()` con 30 mL/min por costumbre. Sus dos argumentos deben ser **los extremos del rango efectivamente validado**, no los que se desea que el sensor mida.

9.3. No presentar Darcy simulado como medición

Mientras TMP y temperatura sean valores supuestos, reemplazar la asignación del resultado por:

```
resultadoDarcy = ResultadoDarcy{};
```

Si se conserva un cálculo de demostración, debe usar una variable distinta y mostrarse como **simulación**. El campo CSV de presión ausente debe quedar vacío o acompañado de un indicador

Simulación. El campo CSV de presión ausente debe quedar vacío o acompañado de un indicador explícito de disponibilidad; cero no debe representar una medición inexistente.

9.4. Ensayos de aceptación de adquisición

Antes de ensayar con la membrana, comprobar:

Conteo conocido: trenes de 1, 10, 50, 100 y 300 Hz, amplitud compatible con 3,3 V y cantidad de pulsos conocida.

Ancho de glitches: pulsos por debajo y por encima de 10 μ s, evitando usar el valor exacto del umbral como frontera ideal.

Overflow: superar 30 000 pulsos y verificar continuidad del conteo acumulado.

Carga de comunicaciones: consultar SCADA y exportar CSV durante el ensayo.

Fallo temporal: forzar una pausa superior a 5 s y verificar el estado de error, no una medición aparentemente normal.

Prueba hidráulica: demostrar sensibilidad real en el extremo inferior del rango validado.

El diagnóstico bruto representa flancos capturables por PCNT, no cualquier perturbación analógica arbitrariamente corta. **Para afirmar integridad eléctrica sigue siendo necesario observar la señal en el pin.**

10. Dictamen final y prioridades

El cierre sigue **RECHAZADO**, aunque se supere una prueba seca de cinco minutos por régimen. Esa prueba no resuelve la sensibilidad a bajo caudal, la incertidumbre, las inconsistencias históricas ni las afirmaciones de seguridad.

Las acciones prioritarias son:

Retirar las garantías no demostradas de seguridad. Verificar ENA, compatibilidad eléctrica y protección de presión. Medir tiempos físicos de parada bajo carga de comunicaciones.

protección de presión: medir tiempos reales de parada bajo carga de comunicaciones.

Adquirir sin ocultar evidencia. Instalar PCNT, conservar conteos brutos y filtrados, estados y duración real de ventana. No convertir anomalías en ceros aparentemente válidos.

Repetir ceros y sensibilidad. Ensayar seco, lleno sin flujo y flujo real, incluyendo transitorios del motor. Si el ruido sobreviviente es indistinguible de señal útil, el firmware no puede cerrar esa brecha.

Calibrar cada sensor con patrón independiente. Suspender la autocalibración por RPM y revisar los factores persistidos en NVS, no solo las constantes compiladas.

Validar el permeado en su rango real. Si la turbina no es repetible a 20 a 100 mL/min, usar la referencia gravimétrica como medición principal sin necesidad de automatizarla inmediatamente.

Completar trazabilidad. Registrar calidad, conteos, pérdidas del buffer, cambios de parámetros, consigna e información de estabilización hidráulica.

Separar control y comunicaciones. Dar al control una ejecución temporal verificable; una tarea periódica mejora la arquitectura, pero no sustituye una protección independiente.

Puede aprobarse por separado la **mejora informática del buffer circular**. No corresponde aprobar aún la **certificación metrológica de ambos caudales**. Si se mantiene el hardware congelado y este no supera las pruebas eléctricas y de sensibilidad, el resultado correcto de la tesis es documentar la limitación y restringir el alcance del instrumento, no ocultarla mediante filtrado.